

Báo cáo Nghiên cứu Kỹ thuật: Đánh giá và Đề xuất Mô hình Nhúng Văn bản Đa ngữ cho Hệ thống RAG Chuyên biệt Lĩnh vực Tuyển dụng (Dự án FANG)

1. Mở đầu và Định hình Ngữ cảnh Kiến trúc Hệ thống FANG

Trong kỷ nguyên của trí tuệ nhân tạo tạo sinh, các hệ thống Phát sinh tăng cường truy xuất (Retrieval-Augmented Generation - RAG) đã thay đổi hoàn toàn cách thức các doanh nghiệp quản trị và khai thác dữ liệu phi cấu trúc. Đối với lĩnh vực tuyển dụng, nơi thông tin ứng viên và yêu cầu công việc được thể hiện dưới dạng các văn bản mang đậm tính chuyên ngành, việc xây dựng một hệ thống RAG đòi hỏi sự tinh tế cực cao trong kiến trúc dữ liệu. Dự án FANG đại diện cho một nỗ lực ở quy mô nền tảng nhằm giải quyết triệt để bài toán tự động hóa quy trình phân tích và đối chiếu Hồ sơ ứng viên (CV/Resume) với Mô tả công việc (Job Description) trong lĩnh vực Công nghệ thông tin (IT). Tuy nhiên, linh hồn của bất kỳ hệ thống RAG nào không nằm ở mô hình ngôn ngữ lớn (LLM) đảm nhiệm việc sinh văn bản, mà nằm ở hệ thống truy xuất thông qua các không gian vector. Báo cáo nghiên cứu này được thiết lập nhằm mục đích phân tích, đối chiếu và đề xuất mô hình nhúng văn bản (text embedding model) tối ưu nhất trong năm 2026, đáp ứng các ràng buộc kỹ thuật khắt khe của Dự án FANG.

Để hiểu được các tiêu chí đánh giá mô hình nhúng, trước tiên cần phải phân tích sâu sắc kiến trúc hệ thống và luồng dữ liệu (Data Flow) đang được vận hành tại FANG. Hệ thống này được chia thành các pha xử lý riêng biệt với mức độ tự động hóa cao. Giai đoạn đầu tiên là quá trình CV Parsing, nơi các tệp định dạng PDF thô được tiếp nhận thông qua các giao diện lập trình ứng dụng (API Endpoint) chuyên biệt.¹ Tại đây, dịch vụ điều phối trung tâm sẽ khởi tạo một luồng xử lý phân tầng (Tiered Processing) vô cùng thông minh nhằm cân bằng giữa tốc độ, độ chính xác và chi phí tài nguyên tính toán. Hệ thống ưu tiên gọi mô hình gemini-flash tại Tầng 1 (Tier 1) như một lực lượng phản ứng nhanh, tận dụng ưu thế về chi phí thấp và độ trễ tối thiểu.¹ Toàn bộ tài liệu PDF được chuyển lên không gian lưu trữ tạm thời của Gemini Files API, sau đó một bộ câu lệnh mẫu (Prompt) mang tính chiến lược được tiêm vào cùng với định dạng lược đồ (Schema) JSON được cấu trúc hóa chặt chẽ thông qua thư viện Pydantic.¹ Trong trường hợp Tầng 1 gặp sự cố do kết nối mạng hoặc nội dung CV vượt quá năng lực phân giải của mô hình nhẹ, hệ thống sẽ tự động kích hoạt Tầng 2 (Tier 2), chuyển giao trọng trách cho mô hình gemini-pro với năng lực nhận thức sâu sắc hơn nhưng đánh đổi bằng chi phí và độ trễ cao hơn.¹ Đầu ra của pha này là sự tách bạch rõ ràng giữa văn bản thô (raw text) và dữ liệu đã được cấu

trúc hóa (JSON), đảm bảo tính toàn vẹn của dữ liệu trước khi lưu trữ vào PostgreSQL thông qua thư viện kết nối bất đồng bộ asyncpg và cơ chế Connection Pool tối ưu hóa hiệu năng.¹

Khi dữ liệu đã sẵn sàng, dự án FANG tiến vào Pha 2 với chiến lược Phân mảnh (Chunking) và Đồng hóa dữ liệu (Ingestion) mang tính đột phá, được định hướng bởi triết lý "Zero-LLM-Cost".¹ Thay vì nhúng trực tiếp dữ liệu JSON gây lãng phí token và phá vỡ cơ chế tập trung (attention mechanism) của các mô hình nhúng, hệ thống thực hiện làm phẳng dữ liệu thành định dạng Markdown. Tại đây, chiến lược Phân mảnh cấu trúc lai (Hybrid Structure-aware Chunking) được kích hoạt, sử dụng bộ chia văn bản dựa trên thẻ tiêu đề (MarkdownHeaderTextSplitter) để bảo toàn tính nguyên tử của từng khối thông tin logic, chẳng hạn như một đoạn mô tả kinh nghiệm làm việc.¹ Đối với các khối vượt quá giới hạn 512 tokens, kiến trúc Truy xuất từ Nhỏ đến Lớn (Small-to-Big Retrieval) được áp dụng. Hệ thống giữ lại tài liệu gốc (Parent Document) làm bối cảnh toàn diện, đồng thời dùng bộ chia ký tự đệ quy (RecursiveCharacterTextSplitter) để băm nhỏ chúng thành các phân mảnh con (Child Chunks) có kích thước dao động từ 150 đến 200 tokens, với mức độ chồng chéo khoảng 20% (tương đương 64 tokens) nhằm duy trì dòng chảy ngữ nghĩa.¹

Đặc biệt, sự xuất hiện của kỹ thuật "Ghim bối cảnh" (Section-Pinning) là một biến số làm thay đổi hoàn toàn cách các mô hình nhúng phải hoạt động.¹ Hệ thống chủ động trích xuất các siêu dữ liệu toàn cục như tên ứng viên, tổng số năm kinh nghiệm, và danh sách kỹ năng cốt lõi để tiêm trực tiếp vào phần tiền tố (prefix) của từng phân mảnh con. Nhờ vậy, một đoạn văn bản cực ngắn mô tả việc "tối ưu hóa truy vấn cơ sở dữ liệu" sẽ mang theo bối cảnh về việc tác giả là một "Kỹ sư Backend với 5 năm kinh nghiệm về Java". Chính những phân mảnh văn bản đậm đặc từ khóa, ngắn gọn (từ 150 - 500 tokens) và giàu ngữ cảnh này sẽ là đầu vào cho mô hình nhúng.

Bối cảnh kiến trúc trên thiết lập những ràng buộc mang tính sống còn đối với việc lựa chọn mô hình nhúng. Thứ nhất, cơ sở dữ liệu PostgreSQL sử dụng phần mở rộng pgvector với chỉ mục đồ thị phân cấp HNSW (Hierarchical Navigable Small World) để lưu trữ và tìm kiếm vector.¹ Để giải quyết bài toán tiêu thụ bộ nhớ RAM khổng lồ của không gian vector, hệ thống kiên quyết áp dụng kiểu dữ liệu lượng tử hóa vô hướng halfvec (float16).¹ Mô hình nhúng được chọn phải duy trì được mức độ thu hồi (Recall) xuất sắc khi bị ép kiểu xuống định dạng 16-bit hoặc phải có số chiều nguyên bản đủ thấp để không làm tràn bộ đệm. Thứ hai, sự giao thoa ngôn ngữ mãnh liệt trong hồ sơ IT (Code-switching) đòi hỏi mô hình phải có khả năng đồng hóa các khái niệm tiếng Anh và tiếng Việt ở một mức độ trừu tượng cao nhất. Cuối cùng, bài toán kinh tế khi phải xử lý hàng triệu phân mảnh dữ liệu buộc chúng ta phải cân nhắc cẩn trọng giữa việc trả phí theo token cho các dịch vụ API thương mại so với chi phí tự duy trì máy chủ cho các mô hình mã nguồn mở. Toàn bộ các khía cạnh này sẽ được giải phẫu chi tiết trong các phần tiếp theo của báo cáo.

2. Động lực học Đa ngữ và Hiện tượng Giao thoa Ngôn ngữ (Code-Switching) trong Miền Dữ liệu IT

Bài toán truy xuất thông tin trong lĩnh vực tuyển dụng nhân sự công nghệ thông tin tại Việt Nam mang một sắc thái ngôn ngữ học cực kỳ đặc thù. Nó không đơn thuần là bài toán dịch thuật một-một (one-to-one translation) mà là bài toán định vị khái niệm trong một không gian ngôn ngữ lai tạp. Sự giao thoa ngôn ngữ (Code-switching) xảy ra với tần suất dày đặc, khi một câu văn có thể chứa cả cấu trúc ngữ pháp tiếng Việt lẫn các danh từ, động từ tiếng Anh đóng vai trò là thuật ngữ chuyên ngành (IT Jargon). Chẳng hạn, một chuyên viên nhân sự (HR) có thể nhập vào thanh tìm kiếm truy vấn: "Cần tìm Kỹ sư phần mềm có kinh nghiệm deploy hệ thống lên đám mây, biết fix bug và tối ưu hóa CI/CD". Trong khi đó, hồ sơ của ứng viên lại được viết hoàn toàn bằng tiếng Anh chuyên nghiệp: "Software Engineer with 5 years of experience in cloud infrastructure deployment, debugging, and CI/CD pipeline optimization".

Để một hệ thống RAG kết nối thành công hai văn bản này, mô hình nhúng không thể dựa vào sự trùng khớp từ vựng (lexical matching) thuần túy. Khả năng truy xuất xuyên ngôn ngữ (Cross-lingual Retrieval Accuracy) trở thành thước đo cốt lõi. Mô hình phải sở hữu một không gian vector ngữ nghĩa (semantic vector space) nơi mà vector đại diện cho "Kỹ sư phần mềm" và vector đại diện cho "Software Engineer" hội tụ tại cùng một cụm tọa độ, bất chấp sự khác biệt hoàn toàn về ký tự và xuất xứ ngôn ngữ.

Năm 2026 đánh dấu một bước nhảy vọt trong khả năng đánh giá các mô hình nhúng đa ngữ tại thị trường Việt Nam với sự xuất hiện của tiêu chuẩn đánh giá VN-MTEB (Vietnamese Massive Text Embedding Benchmark).³ Tiêu chuẩn này bao gồm 41 tập dữ liệu trải dài qua 6 nhóm tác vụ phức tạp: Truy xuất (Retrieval), Tái xếp hạng (Reranking), Phân loại (Classification), Gom cụm (Clustering), Phân loại cặp (Pair Classification), và Tương đồng Ngữ nghĩa (Semantic Textual Similarity).³ Việc quy chuẩn hóa bài kiểm tra này đã lật tẩy nhiều hạn chế của các mô hình trước đây, đồng thời hé lộ những kiến trúc mạng nơ-ron thực sự hiệu quả cho tiếng Việt.

Đáng chú ý, các nghiên cứu chuyên sâu từ báo cáo VN-MTEB chỉ ra rằng có sự phân hóa rõ rệt về hiệu suất dựa trên phương pháp mã hóa vị trí (Positional Encoding) của các mô hình ngôn ngữ lớn làm nền tảng. Các cấu trúc mạng nơ-ron lớn và phức tạp áp dụng kỹ thuật Nhúng vị trí xoay (Rotary Positional Embedding - RoPE) đã chứng minh khả năng vượt trội, đánh bại hoàn toàn các mô hình truyền thống sử dụng Nhúng vị trí tuyệt đối (Absolute Positional Embedding - APE) trong việc nắm bắt các dải ngữ nghĩa dài và cấu trúc ngữ pháp uyển chuyển của tiếng Việt.³ Kiến trúc RoPE cho phép mô hình nhúng hiểu được khoảng cách tương đối giữa các từ thay vì chỉ là vị trí cố định, điều này đặc biệt quan trọng trong tiếng Việt khi cấu trúc cụm từ có thể thay đổi linh hoạt mà không làm biến đổi ý nghĩa toàn cục.

Trong môi trường chuyên biệt của Dự án FANG, khả năng ánh xạ chéo này cần phải vượt qua bài kiểm tra của các thuật ngữ ngách. Những từ viết tắt như "AWS EC2", "K8s", hay "ReactJS" đóng vai trò như những mỏ neo ngữ nghĩa (semantic anchors). Một mô hình nhúng lý tưởng cho FANG phải cân bằng được giữa việc nắm bắt ý nghĩa trừu tượng của toàn bộ câu văn (ví dụ: mô tả về tư duy kiến trúc hệ thống) và việc không làm mờ nhạt đi giá trị của các từ khóa công nghệ này. Sự thất bại của nhiều mô hình nhúng tổng quát (general-purpose embeddings) nằm ở chỗ chúng thường "làm mịn" (smooth out) các từ viết tắt kỹ thuật, biến chúng thành những

vector vô định hình, dẫn đến hiện tượng nhiễu loạn kết quả khi tìm kiếm các kỹ năng đặc thù. Do đó, việc đánh giá các mô hình thương mại và mã nguồn mở trong các phần tiếp theo sẽ được rọi chiếu qua lăng kính cực kỳ khắt khe của sự giao thoa ngôn ngữ và mật độ từ khóa kỹ thuật.

3. Đánh giá Định lượng và Định tính Các Mô hình Nhúng Thương mại (Proprietary Models)

Thị trường cung cấp Giao diện Lập trình Ứng dụng (API) cho các mô hình nhúng vào năm 2026 chịu sự chi phối mạnh mẽ của các tập đoàn công nghệ lớn như Google, OpenAI và Cohere. Mỗi nhà cung cấp mang đến những triết lý thiết kế không gian vector khác biệt, phản ánh qua các chỉ số benchmark đa ngôn ngữ và khả năng xử lý truy vấn hợp.

3.1. Google Multilingual Embeddings: Quyển lực Đa phương thức và Sự tiến hóa của Gemini

Tháng 3 năm 2026, Google đã chính thức phát hành phiên bản xem trước của Gemini Embedding 2, đánh dấu sự khởi đầu của thế hệ mô hình nhúng đa phương thức nguyên bản (all-modality universal).⁴ Kiến trúc mới này tích hợp khả năng xử lý trực tiếp năm loại dữ liệu—văn bản, hình ảnh, video, âm thanh, và tài liệu PDF—vào chung một không gian vector thống nhất với độ dài tiêu chuẩn là 3072 chiều.⁴

Đối với nhu cầu xử lý đa ngôn ngữ, Gemini thể hiện sức mạnh áp đảo. Trong các bài kiểm tra nghiêm ngặt về khả năng truy xuất chéo ngôn ngữ (Cross-lingual Retrieval), nơi các khái niệm văn hóa hoặc thuật ngữ ẩn dụ được mang ra thử thách, biến thể đa ngữ của Gemini đạt chỉ số Recall@1 (R@1) lên đến 0.997, một con số gần như hoàn hảo.⁵ Đối với riêng tiếng Việt, các biến thể trước đó như text-multilingual-embedding-002 (với 768 chiều không gian) đã được ghi nhận là đặc biệt tối ưu cho các tác vụ phi tiếng Anh, vượt qua nhiều đối thủ sừng sỏ về độ chính xác trong việc ánh xạ ngữ nghĩa giữa các văn bản Đông Nam Á.⁶ Khi hệ thống hợp nhất những thành tựu này vào thế hệ Gemini 2, khả năng thấu hiểu sự xen kẽ giữa tiếng Việt và IT Jargon tiếng Anh là không thể phủ nhận.

Tuy nhiên, sức mạnh khổng lồ này mang theo những hệ lụy trực tiếp đến bài toán kiến trúc của FANG. Mặc dù Gemini là "nhà vô địch" trong khả năng xử lý các tài liệu siêu dài (lên đến 32,000 ký tự mà không bị suy giảm khả năng trích xuất "Needle-in-a-Haystack")⁵, nhưng lợi điểm này hoàn toàn bị lãng phí trong hệ thống FANG. Như đã trình bày trong thiết kế kiến trúc, FANG đã chủ động băm nhỏ tài liệu xuống mức 150 - 500 tokens.¹ Việc sử dụng một mô hình được thiết kế để "nhai" 32,000 ký tự cho một đoạn văn bản chỉ vài trăm ký tự giống như việc sử dụng xe tải hạng nặng để giao một lá thư. Đáng lo ngại hơn, Gemini thể hiện hiệu suất cực kỳ tệ hại khi bị ép nén số chiều theo chuẩn Học biểu diễn Matryoshka (Matryoshka Representation Learning - MRL). Các dữ liệu đo lường cho thấy, khi hệ thống cố gắng rút gọn vector 3072 chiều của Gemini xuống mức 256 chiều để tiết kiệm bộ nhớ, hệ số tương quan Spearman (ρ) sụt giảm

thảm hại xuống mức 0.689, xếp bét bảng trong các mô hình thế hệ mới.⁵ Thiết kế nguyên bản của Google không tối ưu cho việc cắt cụt vector (truncation), khiến nó trở thành một cơn ác mộng đối với các cơ sở dữ liệu bị giới hạn nghiêm ngặt về RAM như hạ tầng PostgreSQL của FANG.

3.2. Hệ sinh thái OpenAI: text-embedding-3-large và Bài toán Kinh tế Dữ liệu

Vào đầu năm 2024 và kéo dài ảnh hưởng đến 2026, OpenAI duy trì hai trụ cột chính trong mảng nhúng văn bản: text-embedding-3-small (1536 chiều) và text-embedding-3-large (3072 chiều).⁷ Thế hệ này mang lại sự gia tăng rõ rệt về hiệu suất đa ngôn ngữ so với người tiền nhiệm ada-002 (điểm trung bình trên tập MIRACL đa ngôn ngữ tăng từ 31.4% lên 44.0%).⁷

Được xem là sự lựa chọn mặc định, an toàn cho phần lớn các hệ thống RAG thương mại, text-embedding-3-large cung cấp độ ổn định đáng kinh ngạc. Điểm mạnh lớn nhất của hệ sinh thái OpenAI nằm ở khả năng hỗ trợ nén Matryoshka (MRL) cực kỳ mượt mà. Trái ngược với sự đổ vỡ của Gemini, mô hình large của OpenAI có thể cắt từ 3072 chiều xuống 1024 hoặc thậm chí 256 chiều mà độ suy giảm chất lượng truy xuất gần như không đáng kể (chỉ giảm khoảng 0.6% trên thang đo Spearman, duy trì mức $\rho = 0.762$ tại 256 chiều).⁵ Điều này cho phép các kỹ sư linh hoạt tinh chỉnh kích thước bộ nhớ mà không lo ngại về chất lượng đầu ra. Trong các benchmark chung, dòng mô hình này duy trì điểm MTEB ở mức 64.6.⁸

Dù vậy, khi tiến sâu vào miền dữ liệu chuyên ngành (domain-specific data) và các đánh giá chéo ngôn ngữ phức tạp, ánh hào quang của OpenAI bắt đầu phai nhạt. Các phân tích cho thấy khoảng cách dẫn đầu của OpenAI thu hẹp đáng kể khi rời khỏi vùng an toàn của dữ liệu bách khoa toàn thư (Wikipedia-style data) để đối mặt với các văn bản ngách như tài liệu y tế, pháp lý, hay trong trường hợp của FANG là hồ sơ tuyển dụng IT.⁹ Đối với truy vấn tiếng Việt, các mô hình của OpenAI xử lý ở mức "chấp nhận được" (adequate) thay vì xuất sắc. Hiện tượng này xuất phát từ bản chất của các mô hình tổng quát toàn cầu: chúng có xu hướng đồng nhất hóa các sắc thái khu vực, khiến cho các từ mượn như "Fix bug" hay "Deploy" khi xuất hiện trong câu tiếng Việt đôi khi không được nhóm cụm hoàn hảo với từ gốc tiếng Anh bằng các mô hình được tinh chỉnh cục bộ.

3.3. Cohere Embed-Multilingual v4: Giải pháp cho Dữ liệu Doanh nghiệp Đa ngôn ngữ

Cohere đã xây dựng một danh tiếng vững chắc trong việc phục vụ các luồng dữ liệu doanh nghiệp nhiễu loạn (noisy enterprise data). Biến thể embed-v4 (với 1024 chiều không gian) đạt điểm MTEB đa ngôn ngữ là 65.2⁸, với độ dài ngữ cảnh hỗ trợ lên tới 128,000 tokens.⁸ Trong các bài kiểm thử đối sánh chéo ngôn ngữ khắc nghiệt (Hard Cross-lingual Retrieval), mô hình của Cohere vượt lên dẫn đầu cùng với các đối thủ hàng đầu, đạt điểm Recall@1 trung bình là 0.955.⁴

Một lợi thế đáng kể của Cohere đối với hệ thống FANG là cấu trúc 1024 chiều nguyên bản. Không cần phải thông qua quá trình nén số chiều mang tính đánh đổi như MRL, bản thân vector đầu ra của Cohere đã đủ nhỏ gọn để tiết kiệm một lượng lớn dung lượng lưu trữ trên PostgreSQL so với định dạng 3072 chiều của OpenAI hay Google. Thêm vào đó, khả năng làm sạch các định dạng văn bản bị lỗi, đặc biệt hữu ích khi FANG thực hiện việc chuyển đổi từ dữ liệu JSON bóc tách từ các bản PDF phức tạp sang Markdown, giúp Cohere duy trì được độ chính xác ngay cả khi văn bản đầu vào chứa nhiều ký tự rác.

Bảng 1: So sánh Thông số Kỹ thuật các Mô hình Thương mại (2026)

Tiêu chí Đánh giá	Google Gemini Embedding 2	OpenAI text-embedding-3-large	Cohere embed-v4
Số chiều Nguyên bản (Dims)	3072	3072	1024
Khả năng Nén Matryoshka (256d)	Yếu (ρ giảm xuống 0.689)	Rất mạnh (ρ duy trì 0.762)	Không áp dụng / Không cần thiết
Xử lý Đa phương thức	Có (Văn bản, Hình, Âm thanh, PDF)	Không (Chỉ văn bản)	Hỗ trợ Nhúng Hình ảnh
Hiệu suất Đa ngôn ngữ (MTEB)	Cực cao (R@1 = 0.997)	Khá (MTEB = 64.6)	Tốt (MTEB = 65.2)
Chi phí API (ước tính / 1M tokens)	~\$0.15	\$0.13	\$0.10 - \$0.12
Dữ liệu tham chiếu từ ⁵			

4. Sự Tiến hóa của Mô hình Mã nguồn mở (Open-Source) và Địa phương hóa

Sự dịch chuyển của cộng đồng AI toàn cầu hướng tới các giải pháp mã nguồn mở đã sản sinh ra những kiến trúc có khả năng cạnh tranh sòng phẳng, thậm chí vượt trội hơn các mô hình

thương mại đóng kín, đặc biệt là trong các lĩnh vực yêu cầu sự chuyên biệt hóa sâu sắc. Đối với dự án FANG, việc tích hợp các mô hình này không chỉ giải quyết bài toán tự chủ dữ liệu mà còn mở ra cánh cửa cho việc tối ưu hóa lai tạp (hybrid optimization).

4.1. BAAI/bge-m3: Cỗ máy Cày Aĩ Đa ngôn ngữ (The Multilingual Workhorse)

Được phát triển bởi Viện Trí tuệ Nhân tạo Bắc Kinh (BAAI), bge-m3 nhanh chóng thiết lập tiêu chuẩn vàng mới cho các mô hình nhúng mã nguồn mở. Cái tên "M3" là sự kết tinh của ba triết lý thiết kế cốt lõi¹⁰:

1. **Multi-linguality (Đa ngôn ngữ):** Huấn luyện trên kho dữ liệu khổng lồ bao phủ hơn 100 ngôn ngữ, mang lại khả năng ánh xạ chéo ngôn ngữ ấn tượng.
2. **Multi-granularity (Đa độ hạt):** Khả năng xử lý từ các cụm từ rất ngắn cho đến các tài liệu kéo dài 8192 tokens một cách mượt mà.
3. **Multi-functionality (Đa chức năng):** Đây là đặc điểm biến bge-m3 thành viên ngọc quý. Nó có thể đồng thời sinh ra các vector dày đặc (dense embeddings), vector thưa thớt (sparse retrieval weights, hoạt động tương tự như thuật toán BM25), và biểu diễn đa vector (multi-vector interactions) chỉ trong một lần thực thi tính toán duy nhất.¹⁰

Trong bài kiểm tra chéo ngôn ngữ, mô hình 568 triệu tham số (tương đương khoảng 2.27 GB dung lượng VRAM) này đạt điểm R@1 xuất sắc ở mức 0.940.⁴ Mặc dù trong bài kiểm tra "Kim trong đống bể" (Needle-in-a-Haystack) với các tài liệu siêu dài, hiệu suất của nó bắt đầu có dấu hiệu suy giảm khi vượt qua mốc 8000 ký tự (độ chính xác giảm từ mức hoàn hảo xuống còn 0.920)⁴, nhưng đặc tính này hoàn toàn không ảnh hưởng đến FANG. Vì chiến lược phân mảnh cấu trúc của FANG đã giới hạn chặt chẽ độ dài mỗi đoạn ở mức 150 đến 500 tokens¹, bge-m3 luôn được vận hành trong giới hạn tối ưu nhất của nó, đảm bảo điểm số trích xuất đạt mức 1.000 tuyệt đối.

Với số chiều nguyên bản là 1024, bge-m3 tỏ ra cực kỳ thân thiện với bộ nhớ.¹⁰ Hơn nữa, nó còn thể hiện sức đề kháng khá tốt trước quá trình nén số chiều. Khi bị ép xuống 256 chiều qua kỹ thuật MRL, hệ số tương quan của mô hình vẫn duy trì ở mức 0.744, vượt trội hơn so với Gemini 2 của Google.⁴

4.2. AITeamVN/Vietnamese_Embedding_v2: Sự Tinh chỉnh Chuyên biệt (Localized Fine-Tuning)

Mặc dù bge-m3 sở hữu năng lực xử lý đa ngôn ngữ phi thường, một mô hình được huấn luyện trên quy mô toàn cầu khó có thể nắm bắt trọn vẹn những rẽ nhánh văn hóa và ngữ cảnh vi mô của một vùng địa lý cụ thể. Đó là lúc các nỗ lực tinh chỉnh địa phương (localized fine-tuning) phát huy sức mạnh.

Được phát triển bởi đội ngũ cộng đồng tại Việt Nam, mô hình

AlTeamVN/Vietnamese_Embedding_v2 sử dụng chính nền tảng kiến trúc của BAAI/bge-m3 (cụ thể là từ lõi bge-reranker-v2-m3) và thực hiện tinh chỉnh chuyên sâu trên một bộ dữ liệu đồ sộ bao gồm 1.1 triệu bộ ba (triplets) câu hỏi, tài liệu tích cực và tài liệu tiêu cực (hard negatives) hoàn toàn bằng tiếng Việt.¹² Việc đưa vào các mẫu tiêu cực khó (hard negatives) giúp mô hình rèn luyện khả năng phân biệt những khái niệm có vẻ bề ngoài giống nhau nhưng bản chất khác biệt, một kỹ năng sống còn trong truy xuất tài liệu chuyên ngành.

Kết quả thử nghiệm thực tế trên các tập dữ liệu ngách cho thấy sự lột xác đáng kinh ngạc. Trên cùng một chuẩn đánh giá (Accuracy@1), mô hình BGE-M3 nguyên bản chỉ đạt mức 0.5682, trong khi biến thể được tinh chỉnh Vietnamese_Embedding_v2 đã nhảy vọt lên mức **0.7262**.¹³ Điểm Accuracy@10 cũng tiệm cận mức hoàn hảo với 0.9578 so với 0.8921 của bản gốc.¹³ Sự khác biệt này không đến từ sức mạnh toán học thuần túy, mà đến từ vốn từ vựng học (vocabulary understanding). Bằng cách cho mô hình "tiêu hóa" hàng triệu truy vấn tiếng Việt, nó học được cách thức các từ lóng IT tiếng Anh (IT Jargon) được đan xen một cách tự nhiên vào ngữ pháp tiếng Việt. Khi hệ thống nhận được truy vấn "devops có kinh nghiệm scale hệ thống microservices", mô hình tinh chỉnh này hiểu rõ trọng lượng của cụm từ "scale" và "microservices" không phải là từ ngoại lai vô nghĩa, mà là hạt nhân ngữ nghĩa của toàn bộ câu, qua đó ánh xạ chính xác đến các CV tương ứng. Theo các đánh giá độc lập, biến thể này hiện được công nhận là mô hình nhúng tiếng Việt mạnh mẽ nhất cho các tác vụ truy xuất tổng quát và học thuật tính đến năm 2026.¹²

Bảng 2: So sánh Hiệu suất Mã nguồn mở và Mô hình Tinh chỉnh (2026)

Tiêu chí Đánh giá	BAAI/bge-m3 (Gốc)	AlTeamVN/Vietnamese_Embedding_v2	Qwen3-Embedding-0.6B
Kích thước Mô hình	568 Triệu tham số	568 Triệu tham số (Tinh chỉnh)	600 Triệu tham số
Số chiều Vector (Dims)	1024	1024	Tùy chỉnh (32 - 1024)
Kiến trúc Sinh Vector	Dense, Sparse, Multi-vector	Dense	Dense
Độ chính xác	0.5682	0.7262	Không có dữ liệu

(Accuracy@1) (Tập chuyên Việt)			
Điểm mạnh cốt lõi	Tính đa chức năng (M3), Chéo ngữ rộng	Tối ưu tuyệt đối cho tiếng Việt lai tạp	Trọng lượng nhẹ, Đa ngữ toàn cầu
Dữ liệu tham chiếu từ ¹³			

5. Sự Cộng sinh Giữa Chiến lược Phân mảnh (Chunking) và Thuật toán Truy xuất Lai (Hybrid Retrieval)

Mô hình nhúng không tồn tại trong khoảng không; hiệu quả của chúng bị ràng buộc chặt chẽ bởi cách tài liệu được tiền xử lý và cắt gọt. Chiến lược phân mảnh của Dự án FANG không đi theo lối mòn truyền thống. Nó triển khai kỹ thuật Phân mảnh cấu trúc lai (Hybrid Structure-aware Chunking) với cốt lõi là kiến trúc Small-to-Big Retrieval và Section-Pinning.¹ Mọi khối dữ liệu (Parent Document) vượt quá 512 tokens sẽ được lưu trữ độc lập, trong khi không gian vector chỉ chứa các phân mảnh con (Child Chunks) dung lượng từ 150 đến 200 tokens (với 20% độ chồng chéo).¹ Hơn nữa, các đoạn mã metadata quan trọng (Tên ứng viên, năm kinh nghiệm, kỹ năng) được tiêm vĩnh viễn vào đầu mỗi phân mảnh con.¹

Sự lựa chọn kiến trúc này tạo ra một hệ quả sinh thái:

- Chấm dứt cuộc chạy đua "Ngữ cảnh Dài" (Long-context Race):** Các mô hình tự hào với ngữ cảnh 32K hay 128K tokens như Cohere v4 hay Gemini 2 trở nên thừa thãi. Hệ thống FANG yêu cầu mô hình phải biểu diễn xuất sắc tính toàn vẹn cục bộ (local integrity) của đoạn văn bản ngắn, đồng thời đánh trọng số chính xác cho phần siêu dữ liệu được tiêm vào (Section-Pinning).
- Sức mạnh của Sparse Search trong Truy xuất Lai (Hybrid Search):** FANG quy định việc sử dụng đa luồng bằng thuật toán Reciprocal Rank Fusion (RRF), tổng hợp điểm số từ tìm kiếm ngữ nghĩa (Dense Vector Search) và tìm kiếm từ khóa thưa thớt (Sparse Search - BM25).¹ Đây là một quyết định kiến trúc mang tầm nhìn chiến lược. Trong tuyển dụng IT, Dense Vector xuất sắc trong việc hiểu rằng "Front-end Developer" tương đương với "Web UI Engineer". Tuy nhiên, Dense Vector lại thường thất bại thảm hại trước các chuỗi ký hiệu kỹ thuật mang tính quyết định, ví dụ như sự khác biệt giữa framework "Angular 14" và "Angular 15", hay mã sản phẩm "AWS EC2" so với "AWS S3". Tại đây, thuật toán BM25 (chấm điểm dựa trên tần suất từ khóa) sẽ nhảy vào cứu nguy, đảm bảo các thuật ngữ

(Jargon) này được khớp chính xác từng ký tự.

Chính tại giao điểm này, **BAAI/bge-m3 và các biến thể của nó tỏa sáng rực rỡ**. Nhờ kiến trúc được thiết kế theo phong cách ColBERT, bge-m3 là một trong số ít các nền tảng có khả năng xuất ra song song cả vector mảng đặc (cho Dense Search) và trọng số từ vựng (lexical weights cho Sparse Search) chỉ thông qua một lượt suy luận duy nhất (single inference pass).¹¹ Thay vì hệ thống FANG phải vận hành một API gọi đến OpenAI để lấy vector, rồi tiếp tục khởi tạo một bộ phân tích cú pháp mã nguồn mở chạy nền để tính toán điểm số BM25, việc sử dụng bge-m3 gom cả hai tác vụ vào chung một đường ống. Điều này tiết kiệm đáng kể chu kỳ vi xử lý (CPU cycles), loại bỏ chi phí duy trì hai hệ thống tách biệt và cung cấp dữ liệu đầu vào hoàn hảo để hệ thống RRF tính toán ra kết quả ứng viên chính xác nhất.

6. Giải phẫu Giới hạn Hệ thống: PostgreSQL, Chỉ mục HNSW và Ma thuật Lượng tử hóa halfvec

Rào cản vật lý lớn nhất của Dự án FANG không nằm ở chất lượng AI, mà nằm ở giới hạn của bộ nhớ RAM máy chủ cơ sở dữ liệu. Với việc sử dụng PostgreSQL kết hợp cùng phần mở rộng pgvector và thuật toán chỉ mục HNSW (Hierarchical Navigable Small World), mọi sai lầm trong việc quản trị chiều kích vector sẽ dẫn đến sự sụp đổ của hệ thống lưu trữ.¹

6.1. Giải mã Nút thắt Cổ chai của Chỉ mục HNSW

Thuật toán HNSW hoạt động bằng cách xây dựng một mạng lưới đồ thị nhiều tầng (multi-layer graph) kết nối các điểm dữ liệu (vector). Nó mang lại tốc độ truy vấn cực nhanh (chỉ từ vài đến vài chục mili-giây) nhưng lại ngốn một lượng RAM khổng lồ, bởi toàn bộ cấu trúc đồ thị này cần được tải vào bộ nhớ đệm dùng chung (Shared Buffers) của PostgreSQL để tránh các lệnh gọi I/O từ ổ cứng tốn kém.¹⁷

Nghiêm trọng hơn, pgvector tồn tại một giới hạn kiến trúc mã nguồn cốt lõi (hard-coded limit) đối với HNSW: **Nếu sử dụng kiểu dữ liệu vector tiêu chuẩn (dạng số thực 32-bit/float32), HNSW không thể lập chỉ mục cho các vector có số chiều vượt quá 2000.**² Điều này tạo ra một rào cản chí mạng đối với các mô hình thương mại hàng đầu. Cả text-embedding-3-large của OpenAI và Gemini Embedding 2 của Google đều mặc định trả về không gian vector **3072 chiều**.⁴ Nếu các kỹ sư của FANG cố gắng đưa cấu trúc 3072 chiều này vào lệnh CREATE INDEX USING hnsw, PostgreSQL sẽ trực tiếp báo lỗi từ chối thao tác (ERROR: column cannot have more than 2000 dimensions for hnsw index).²

Để lách qua khe cửa hẹp này, các hệ thống buộc phải nhờ đến các kỹ thuật ép dài.

6.2. Cuộc Cứu rỗi mang tên halfvec (Float16 Scalar Quantization)

Kể từ bản phát hành 0.7.0, pgvector đã mang đến một phép màu kỹ thuật mang tên Lượng tử hóa vô hướng (Scalar Quantization), hiện thân qua kiểu dữ liệu halfvec.¹⁷ Thay vì sử dụng 32 bit

(4 bytes) để biểu diễn một con số tọa độ, halfvec chỉ sử dụng 16 bit (2 bytes).¹⁹ Về mặt cấu trúc nhị phân, float32 gồm 1 bit dấu, 8 bit lũy thừa (exponent) và 23 bit phần trị (mantissa). Khi bị cắt xuống float16, cấu trúc trở thành 1 bit dấu, 5 bit lũy thừa và 10 bit phần trị.¹⁹

Lợi ích của sự đánh đổi này là khổng lồ:

- **Giải phóng Giới hạn HNSW:** Việc ép xuống 16-bit giúp nhân đôi giới hạn lập mục lục của HNSW từ 2000 chiều lên **4000 chiều**, cho phép PostgreSQL nuốt trọn vector 3072 chiều của OpenAI hay Gemini mà không báo lỗi.²
- **Chiến thắng Bài toán Dung lượng:** Kích thước cốt lõi của không gian lưu trữ và chỉ mục giảm chính xác 50%. Một cột vector nặng 60GB sẽ lập tức teo nhỏ còn 30GB.²⁰
- **Gia tốc Độ trễ:** Bằng cách nhét vừa gấp đôi số lượng vector vào trong một trang nhớ 8KB tiêu chuẩn của PostgreSQL, hệ thống giảm thiểu triệt để hiện tượng tràn bộ nhớ đệm (page eviction). Thử nghiệm thực tế cho thấy tốc độ xây dựng chỉ mục (index build time) tăng lên hơn gấp đôi (từ 377 giây xuống 163 giây cho một tập dữ liệu chuẩn), trong khi số lượng truy vấn trên giây (QPS) được đẩy lên cao hơn (từ 627 lên 642 QPS).¹⁷

Vậy cái giá phải trả cho độ chính xác là gì? Một định kiến phổ biến cho rằng việc cắt xén phần lẻ thập phân sẽ làm méo mó nghiêm trọng vị trí của vector trong không gian. Tuy nhiên, trong không gian toán học có số chiều siêu cao (high-dimensional geometry), khoảng cách tương đối (cosine similarity) giữa các tài liệu không bị suy chuyển bởi những sai số nhỏ bé ở phân lớp thập phân thứ ba hay thứ tư (giữa 0.000123 và 0.000124).¹⁹ Các nghiên cứu độc lập khẳng định rằng, sau khi lượng tử hóa xuống halfvec, tỷ lệ sụt giảm độ bao phủ truy xuất (Recall Degradation) gần như bằng 0, luôn được duy trì ở mức thiệt hại dưới 1% so với nguyên bản float32 (ví dụ: Recall@10 giữ vững ở mức 0.945 trong cả hai bài test).¹⁹ Quyết định triển khai halfvec của dự án FANG là một chiến lược phòng thủ kiến trúc xuất sắc.

6.3. Giải pháp Nén Matryoshka (MRL) và Sự Lựa chọn Kích thước Nguyên bản

Mặc dù halfvec giúp chứa được 3072 chiều của OpenAI, nhưng nó vẫn tiêu tốn tới khoảng 6.1 Kilobytes (3072 * 2 bytes) cho mỗi vector. Ở quy mô hàng chục triệu chunks, con số này vẫn tạo áp lực khủng khiếp lên RAM.

OpenAI cho phép cắt ngắn vector (truncation) thông qua API để giảm xuống 256 chiều nhờ công nghệ MRL. Khi ép thêm bằng halfvec, mỗi vector chỉ còn tốn khoảng 512 Bytes. Tuy nhiên, điều này đồng nghĩa với việc FANG phải trả tiền cho một model đắt đỏ có khả năng sinh ra 3072 chiều, sau đó vứt bỏ 90% lượng thông tin đó đi, dù sự sụt giảm độ chính xác (decay) của OpenAI là khá thấp (chỉ mất khoảng 0.6% chất lượng).⁵

Ngược lại, mô hình BAAI/bge-m3 hoặc Vietnamese_Embedding_v2 sở hữu không gian vector nguyên bản gọn gàng ở mức **1024 chiều**.¹⁰ Khi áp dụng định dạng halfvec, mỗi vector sẽ tiêu tốn chính xác **2048 Bytes (2 KB)**. Nếu hệ thống FANG mở rộng quy mô lên 10 triệu chunks vẫn

bản, không gian lưu trữ thô cho vector chỉ tốn khoảng **20 GB RAM** — một con số lý tưởng, hoàn toàn có thể chạy mượt mà trên một máy chủ đám mây tầm trung mà không cần cắt xén bất kỳ dữ liệu biểu diễn ngữ nghĩa nào.

6.4. Bóng ma của Quá trình Suy giảm Chỉ mục HNSW (Index Decay)

Một cảnh báo cực kỳ quan trọng đối với hạ tầng FANG: Chỉ mục HNSW không phải là một thực thể tĩnh. Khi khối lượng lớn CV liên tục được nạp (ingest), cập nhật, hoặc xóa bỏ, cấu trúc đồ thị bên trong cơ sở dữ liệu sẽ dần bị phình to (bloat), phân mảnh, và dẫn đến hiện tượng **"Suy giảm thu hồi ngầm" (Silent Recall Degradation)**.²² Các vector bị mờ côi hoặc đồ thị bị lệch cấu trúc sẽ khiến hệ thống không thể tìm thấy những CV tốt nhất dù chúng vẫn nằm trong DB. Để giải quyết, đội ngũ DBA của FANG không thể chỉ dựa vào việc bảo trì định kỳ một cách thụ động, mà phải thiết lập hệ thống giám sát sức khỏe phân mảnh chỉ mục, và thực thi các lệnh REINDEX INDEX CONCURRENTLY một cách có chiến lược vào các thời điểm máy chủ nhàn rỗi để định hình lại đồ thị.¹⁸

7. Bài toán Kinh tế Vi mô: Phân tích TCO (Total Cost of Ownership) Giữa Trả phí API và Tự Lưu trữ (Self-Hosting)

Quyết định lựa chọn mô hình không chỉ xoay quanh độ chính xác, mà còn là một bài toán sinh tồn về tài chính và độ trễ ở cấp độ hệ thống sản xuất (production). Hệ thống RAG chuyên biệt như FANG không hoạt động trong không gian tĩnh. Kiến trúc liên tục tiến hóa, đồng nghĩa với việc kho dữ liệu phải được băm lại (re-embed) nhiều lần trong vòng đời phát triển.⁸

7.1. Phân tích Dòng tiền của Mô hình Thương mại (API Costs)

Các dịch vụ cung cấp API thu phí dựa trên đơn lượng 1 triệu tokens. Mức giá tham khảo năm 2026:

- **OpenAI text-embedding-3-large:** \$0.13 / 1 triệu tokens.²³
- **Cohere embed-v4:** \$0.10 / 1 triệu tokens.⁸
- **Google Gemini 2.5 Flash:** ~\$0.10 - \$0.30 / 1 triệu tokens (phụ thuộc vào tier và loại dữ liệu).²⁵

Giả lập một bài toán ở quy mô vừa phải cho FANG: Hệ thống đang lưu trữ 1 triệu Hồ sơ CV và Mô tả công việc (JD). Trung bình, mỗi CV sau khi đi qua cấu trúc Small-to-Big Retrieval sẽ tạo ra khoảng 20 phân mảnh (chunks). Mỗi phân mảnh mang theo metadata được ghim có khối lượng khoảng 250 tokens.

Tổng khối lượng token cần xử lý: $1,000,000 \text{ CVs} \times 20 \text{ Chunks} \times 250 \text{ Tokens} = 5 \text{ Tỷ Tokens}$.

Chi phí khởi tạo ban đầu để nạp dữ liệu (Ingestion) qua OpenAI text-embedding-3-large sẽ tốn

khoảng **\$650 USD**.⁹ Thoạt nhìn, đây là một con số có vẻ rẻ mạt so với việc mua sắm máy chủ. Tuy nhiên, *Cái bẫy API* nằm ở chi phí ngấm của quá trình Re-embedding. RAG là một vòng lặp tinh chỉnh liên tục. Khi FANG cập nhật cách trích xuất kỹ năng, thay đổi chiến lược chunking (ví dụ: mở rộng từ 250 lên 300 tokens), hoặc chuyển sang dùng thử một mô hình mới, toàn bộ 5 tỷ tokens kia phải được băm lại và tính tiền lại từ đầu.⁸ Vòng lặp phát triển nhanh Agile có thể đòi hỏi re-embed 2-3 lần một tháng, khiến hóa đơn định kỳ tăng vọt lên hàng ngàn USD, biến một chi phí "rẻ" thành một khoản thuế duy trì nền tảng vĩnh viễn.

Ngoài ra, việc gửi 10 triệu lệnh gọi API (requests) qua mạng internet tạo ra rủi ro khủng khiếp về độ trễ (Network Latency Bottleneck). Đội ngũ FANG sẽ phải đối mặt với các bài toán đau đầu về quản lý băng thông, xử lý các lỗi giới hạn tốc độ (Rate Limits - 429 Too Many Requests), và cài đặt các hàng đợi bất đồng bộ phức tạp (Exponential Backoff).²⁷

Hơn thế nữa, từ góc độ bảo mật, Hồ sơ ứng viên (CV) là tài nguyên chứa đựng Thông tin Nhận dạng Cá nhân (PII) cực kỳ nhạy cảm. Việc liên tục bơm dữ liệu riêng tư của người dùng ra các điểm cuối (endpoints) bên ngoài lãnh thổ tổ chức sẽ vấp phải các rào cản pháp lý và bộ tiêu chuẩn bảo mật chủ quyền dữ liệu (Data Sovereignty).²⁸

7.2. Hiệu quả Năng lượng của Mô hình Mã Nguồn Mở Tự lưu trữ (Self-Hosting)

Lựa chọn lưu trữ cục bộ các mô hình như BGE-M3 hoặc Vietnamese_Embedding_v2 giải quyết triệt để ba rủi ro: Chi phí tái băm (Re-embedding), Độ trễ mạng (Network Latency), và Rò rỉ dữ liệu (Data Privacy).⁹

- **Zero API Cost:** Bằng cách tự host, chi phí gọi hàm nhúng là \$0. Các kỹ sư của FANG có thể thoải mái thử nghiệm, re-embed hàng tỷ token mỗi đêm mà không phải lo lắng về hóa đơn.⁹
- **Cấu hình Phần cứng Linh hoạt:** Với quy mô 568 triệu tham số, dòng họ mô hình BGE-M3 có kích thước VRAM vô cùng khiêm tốn. Khi áp dụng các cơ chế chạy nội suy với độ chính xác float16 (hoặc bfloat16), mô hình chỉ chiếm dụng vốn vẹn **1.06 GB VRAM** không gian bộ nhớ đồ họa.²⁹ Khối lượng siêu nhẹ này có nghĩa là FANG không cần đầu tư các siêu máy tính đắt đỏ như NVIDIA A100 hay H100. Chỉ với các dòng card GPU phổ thông, chi phí thấp như NVIDIA RTX 3060/3090, hoặc các cụm máy chủ đám mây thuê ngoài sử dụng GPU NVIDIA T4/L40S, hệ thống đã có thể vận hành ổn định với thông lượng (throughput) xử lý khổng lồ, lên tới hàng nghìn vectors mỗi giây (batch processing).²⁸
- **Độ trễ cục bộ (Local Latency):** Vì ứng dụng FastAPI (main.py của FANG) và mô hình nhúng chia sẻ chung một cơ sở hạ tầng mạng cục bộ (LAN hoặc IPC), thời gian truyền tải dữ liệu gần như bằng 0. Tốc độ nhúng giờ đây chỉ phụ thuộc vào năng lực xử lý số học của GPU nội bộ, mang lại một đường ống Ingestion với tốc độ ánh sáng.

8. Tổng kết và Đề xuất Kiến trúc Định hướng

Từ những phân tích đa chiều — bao phủ từ tính đặc thù ngôn ngữ học của IT Jargon tiếng Việt, đến các ràng buộc về bộ nhớ của PostgreSQL, và sự khắc nghiệt của bài toán kinh tế hệ thống — Báo cáo Nghiên cứu Kỹ thuật đưa ra các đề xuất thiết kế kiến trúc như sau đối với Dự án FANG:

1. Quyết định Mô hình Nhúng Cốt lõi:

Khuyến nghị loại bỏ việc sử dụng các mô hình thương mại dạng API (OpenAI text-embedding-3-large, Google Gemini Embedding 2) làm lựa chọn ưu tiên. Mặc dù sở hữu những sức mạnh học thuật đáng nể trong không gian tổng quát, chúng bộc lộ quá nhiều nhược điểm chí mạng khi áp dụng vào FANG: Kích thước 3072 chiều mặc định gây quá tải cho HNSW, Gemini 2 nén MRL quá kém, và chi phí tái băm (re-embedding) liên tục tạo thành một lỗ đen tài chính. Quan trọng hơn, việc chuyển PII của ứng viên ra ngoài vi phạm các nguyên tắc bảo mật.

Khuyến nghị triển khai mô hình mã nguồn mở Tự lưu trữ (Self-hosted) thuộc họ BGE-M3, đặc biệt là biến thể tinh chỉnh AITeamVN/Vietnamese_Embedding_v2.

Sự lựa chọn này đáp ứng hoàn hảo từng mảnh ghép trong kiến trúc của FANG:

- Khả năng ánh xạ "Code-switching" giữa thuật ngữ IT tiếng Anh và văn phong tiếng Việt được gia tăng 28% độ chính xác (Accuracy) nhờ việc huấn luyện trực tiếp trên dữ liệu nội địa với các mẫu tiêu cực khó (Hard Negatives).
- Kiến trúc đa chức năng (Multi-functionality) sản sinh đồng thời vector mảng đặc (Dense Semantic) và trọng số từ vựng (Sparse Lexical) mà không tiêu tốn thêm tài nguyên. Điều này khớp nối tuyệt đối với chiến lược truy xuất lai (Hybrid Retrieval) qua RRF, giúp phát hiện chính xác các thuật ngữ công nghệ ngách như "AWS EC2" mà không bị các vector ngữ nghĩa làm nhòe.

2. Đề xuất Kiến trúc Cơ sở dữ liệu và Không gian Vector:

- Sử dụng định dạng **1024 chiều** nguyên bản của mô hình. Cấu hình cột lưu trữ trên PostgreSQL với phần mở rộng pgvector bắt buộc ở kiểu dữ liệu lượng tử hóa **halfvec (float16)**. Việc này ép dung lượng mỗi vector xuống mức 2KB, giảm thiểu 50% nhu cầu RAM (tiết kiệm không gian lưu trữ từ 60GB xuống 30GB cho các cụm dữ liệu quy mô vừa) và tăng gấp đôi tốc độ xây dựng chỉ mục HNSW với mức hao hụt độ chính xác chưa đến 1%.
- Triển khai hệ thống tự động giám sát độ phình (bloat) của các node trong HNSW graph, cài đặt các quy trình chạy nền tự động kích hoạt lệnh REINDEX INDEX CONCURRENTLY vào các khung giờ thấp điểm để ngăn chặn hiện tượng suy giảm thu hồi ngầm (silent recall degradation).

Bằng cách tích hợp sâu sát những giải pháp kỹ thuật tinh giản nhưng mạnh mẽ này, Dự án FANG sẽ thiết lập được một pháo đài kiến trúc vững chắc, tối ưu hóa toàn diện hiệu suất vòng lặp RAG, đồng thời duy trì khả năng kiểm soát tài chính và bảo mật ở mức độ tuyệt đối.

Nguồn trích dẫn

1. system_architecture.md
2. pgvector, a guide for DBA - Part 2: Indexes (update march 2026) - dbi services, truy cập vào tháng 4 8, 2026, <https://www.dbi-services.com/blog/pgvector-a-guide-for-dba-part-2-indexes-update-march-2026/>
3. VN-MTEB: Vietnamese Massive Text Embedding Benchmark - ACL Anthology, truy cập vào tháng 4 8, 2026, <https://aclanthology.org/2026.findings-eacl.86.pdf>
4. Which Embedding Model Should You Actually Use in 2026? I Benchmarked 10 Models to Find Out - DEV Community, truy cập vào tháng 4 8, 2026, https://dev.to/chen_zhang_bac430bc7f6b95/which-embedding-model-should-you-actually-use-in-2026-i-benchmarked-10-models-to-find-out-58bc
5. Best Embedding Model for RAG 2026: 10 Models Compared - Milvus Blog, truy cập vào tháng 4 8, 2026, <https://milvus.io/blog/choose-embedding-model-rag-2026.md>
6. Text embeddings API | Generative AI on Vertex AI - Google Cloud Documentation, truy cập vào tháng 4 8, 2026, <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/model-reference/text-embeddings-api>
7. New embedding models and API updates - OpenAI, truy cập vào tháng 4 8, 2026, <https://openai.com/index/new-embedding-models-and-api-updates/>
8. Best Embedding Models for RAG (2026): Ranked by MTEB Score, Cost, and Self-Hosting, truy cập vào tháng 4 8, 2026, <https://blog.prem.ai.io/best-embedding-models-for-rag-2026-ranked-by-mteb-score-cost-and-self-hosting/>
9. OpenAI text-embedding-3-large vs bge-m3 vs Zembed-1: My Comparison - Reddit, truy cập vào tháng 4 8, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rlkhjk/openai_textembedding_3large_vs_bgem3_vs_zembed1_my/
10. BGE-M3 — BGE documentation - BGE Models, truy cập vào tháng 4 8, 2026, https://bge-model.com/bge/bge_m3.html
11. BAAI/bge-m3 - Hugging Face, truy cập vào tháng 4 8, 2026, <https://huggingface.co/BAAI/bge-m3>
12. Which Works Best for Vietnamese? A Practical Study of Information Retrieval Methods across Domains - ACL Anthology, truy cập vào tháng 4 8, 2026, <https://aclanthology.org/2026.findings-eacl.110v2.pdf>
13. AITeamVN/Vietnamese_Embedding_v2 - Hugging Face, truy cập vào tháng 4 8, 2026, https://huggingface.co/AITeamVN/Vietnamese_Embedding_v2
14. README.md · AITeamVN/Vietnamese_Embedding at main - Hugging Face, truy cập vào tháng 4 8, 2026, https://huggingface.co/AITeamVN/Vietnamese_Embedding/blob/main/README.md
15. Which Works Best for Vietnamese? A Practical Study of Information Retrieval Methods across Domains - ACL Anthology, truy cập vào tháng 4 8, 2026,

- <https://aclanthology.org/2026.findings-eacl.110.pdf>
16. The Best Open-Source Embedding Models in 2026 - BentoML, truy cập vào tháng 4 8, 2026, <https://www.bentoml.com/blog/a-guide-to-open-source-embedding-models>
 17. What's new in pgvector v0.7.0 - Supabase, truy cập vào tháng 4 8, 2026, <https://supabase.com/blog/pgvector-0-7-0>
 18. pgvector Index Selection: IVFFlat vs HNSW for PostgreSQL Vector Search | by Philip McClarence | Apr, 2026 | Medium, truy cập vào tháng 4 8, 2026, <https://medium.com/@philmcc/pgvector-index-selection-ivfflat-vs-hnsw-for-postgresql-vector-search-6eff26aaa90c>
 19. halfvec: Half the Bits, Twice the speed? - DEV Community, truy cập vào tháng 4 8, 2026, https://dev.to/abhishek_gautam-01/halfvec-half-the-bits-twice-the-speed-3506
 20. Scaling pgvector: Memory, Quantization, and Index Build Strategies - DEV Community, truy cập vào tháng 4 8, 2026, https://dev.to/philip_mcclarence_2ef9475/scaling-pgvector-memory-quantization-and-index-build-strategies-8m2
 21. The Case Against PGVector | Hacker News, truy cập vào tháng 4 8, 2026, <https://news.ycombinator.com/item?id=45798479>
 22. Signal-driven health monitoring for HNSW indices w/ pgvector | by Jake Casto - Medium, truy cập vào tháng 4 8, 2026, <https://medium.com/engineering-layers/signal-driven-health-monitoring-for-hnsw-indices-w-pgvector-ba35d9a6e575>
 23. Best Embedding Models 2026 - 6 Tested for RAG & Search | PE Collective, truy cập vào tháng 4 8, 2026, <https://pecollective.com/tools/best-embedding-models/>
 24. Embedding Models Comparison 2026: OpenAI vs Cohere vs Voyage vs BGE - Reintech, truy cập vào tháng 4 8, 2026, <https://reintech.io/blog/embedding-models-comparison-2026-openai-cohere-voyage-bge>
 25. Gemini Pricing in 2026 for Individuals, Orgs & Developers - Finout, truy cập vào tháng 4 8, 2026, <https://www.finout.io/blog/gemini-pricing-in-2026>
 26. Google Gemini API Pricing 2026: Complete Cost Guide per 1M Tokens - MetaCTO, truy cập vào tháng 4 8, 2026, <https://www.metacto.com/blogs/the-true-cost-of-google-gemini-a-guide-to-api-pricing-and-integration>
 27. We Benchmarked 20+ Embedding APIs with Milvus: 7 Insights That Will Surprise You, truy cập vào tháng 4 8, 2026, <https://milvus.io/blog/we-benchmarked-20-embedding-apis-with-milvus-7-insights-that-will-surprise-you.md>
 28. Migrate OpenAI to Self-Hosted LLM: 2026 Strategic Guide - Emerline, truy cập vào tháng 4 8, 2026, <https://emerline.com/blog/migrate-openai-self-hosted-llm-guide>
 29. BAAI/bge-m3 · [AUTOMATED] Model Memory Requirements - Hugging Face, truy cập vào tháng 4 8, 2026, <https://huggingface.co/BAAI/bge-m3/discussions/64>
 30. What it is and how to install and run locally the LLM AI: bge-m3 - Makiai, truy cập

vào tháng 4 8, 2026, <https://makiai.com/en/what-it-is-and-how-to-install-and-run-locally-the-llm-ai-bge-m3/>

31. Vector Database Comparison 2026: Pinecone vs pgvector vs Chroma vs Weaviate, truy cập vào tháng 4 8, 2026, <https://www.groovyweb.co/blog/vector-database-comparison-2026>